

Punto 5 — Plan de 7 días

Valdris: El Nucleo Profundo

17 mayo — 24 mayo 2026. Entrega final: 28 mayo

Marino Rodriguez	Motor del juego — logica, BFS, IA, TurnManager
Ventura Pacheco	Persistencia JSON, tests JUnit, integracion
Marcos Castro	JavaFX — UI, renderizado, eventos, MVC

- * Dias 18 y 19 de mayo: ritmo reducido (clases). El resto de dias: dedicacion maxima.
- * IA como copiloto permanente: generar codigo, revisar, debugear, integrar. No copiar ciegamente.
- * Todo el codigo de estructuras de datos (LSE, Cola, Grafo) se REUTILIZA del repositorio anterior.
- * Objetivo del 24: juego jugable y demostrable. Objetivo del 28: memoria + video + pulido final.
- * Recorte pactado: 12 salas (no 34), 4 subtipos de enemigo (no 8), 6 armas (no 12). El sistema es el mismo.

5.1 Vision general del plan

- * El plan tiene DOS fases: Fase A (días 1-5, motor + UI basica) y Fase B (días 6-7, integracion + pulido).
- * Marino y Pacheco trabajan en paralelo desde el día 1. Castro se une al motor desde el día 3.
- * Cada día tiene una tarea principal y un criterio de "hecho" concreto. Si no se cumple, se ajusta al día siguiente.
- * Los tests se escriben EN PARALELO con el código, no al final. Pacheco lidera esto desde el día 1.
- * La memoria y el video se hacen los días 25-27 una vez el código esta cerrado.

Resumen ejecutivo — 7 días

Día	Fecha	Marino (Motor)	Pacheco (Tests/JSON)	Castro (JavaFX)	Entregable del día
1	17 may	Clases base + BFS	Setup proyecto + tests LSE/Grafo	Ventana principal + Grid	Proyecto compila. Grid vacio en pantalla.
2	18 may	TurnManager fase 1	Tests BFS + Cell	Renderizado de sala	Sala se muestra con celdas. BFS testeado.
3	19 may	Sistema de items	Tests TurnManager	Jugador visible + click	Jugador se mueve en pantalla.
4	20 may	IA enemiga (3 tipos)	Tests IA + GameState	Panel lateral + UI combate	Combate jugable contra un enemigo.
5	21 may	Serializacion JSON completa	Tests JSON + integracion	Inventario + pociones UI	Guardar y cargar partida funciona.
6	22 may	Generacion mapa (12 salas)	Tests mapa + items	Transicion entre salas	Juego completo de punta a punta.
7	23 may	Pulido + bugs criticos	Cobertura tests >= 70%	Pantallas finales	Version jugable y demostrable.

Amarillo = día con ritmo reducido (clases).

5.2 Desglose detallado por día

DIA 1 — 17 mayo — Sabado		Ritmo: COMPLETO
Marino	Pacheco	Castro
- Crear estructura de paquetes del proyecto	- Setup IntelliJ: modulos, JUnit5, Gson	- Crear proyecto JavaFX con estructura MVC
- Clases base: Unit, Player, Enemy (herencia)	- Copiar LSE, Cola, Grafo del repo anterior	- MainApp.java: ventana principal 1280x720
- Cell (tipo, contenido, isWalkable())	- Tests unitarios de Cell.isWalkable()	- GameView: GridPane para la sala (canvas)
- Room (matriz de Cell, dimensiones)	- Tests de BFS: celdas alcanzables correctas	- Renderizar sala vacia con celdas de colores
- BFS de movimiento en Room (getCellsInRange())	- Tests de Room: creacion y acceso a celdas	- Distinguir visualmente WALL, FLOOR, DOOR
- Reutilizar Grafo del repo anterior	- Verificar que el Grafo del repo funciona	
ENTREGABLE: El proyecto compila sin errores. La ventana JavaFX muestra un grid de celdas coloreadas. BFS con tests pasando.		

DIA 2 — 18 mayo — Domingo		Ritmo: REDUCIDO
Marino	Pacheco	Castro
- TurnManager: estructura de fases (enum Phase)	- Tests de TurnManager: transicion de fases	- GameController: capturar clicks en celdas del grid
- Fase 1 — Movimiento: mover jugador a celda valida	- Tests de movimiento valido e invalido	- Resaltar celdas alcanzables por BFS (color verde)
- Integrar BFS con TurnManager	- Tests de Item.use() con pocion simple	- Mover jugador visualmente al hacer click
- Clase Item base con use(Player p)	- Documentar estructura de clases en UML basico	- Sprite simple del jugador (rectangulo de color)
ENTREGABLE: El jugador se mueve en pantalla haciendo click en celdas validas. Las celdas alcanzables se resaltan.		

! Dia reducido: priorizar movimiento del jugador funcional sobre todo lo demas.

DIA 3 — 19 mayo — Lunes		Ritmo: REDUCIDO
Marino	Pacheco	Castro
- Sistema de items: Weapon, Armor, Potion (herencia)	- Tests de inventario: equipar, usar, descartar	- Panel lateral derecho: HP, nombre, zona actual
- Inventario del jugador: LSE + slots equipo	- Tests de afinidad de armas por personaje	- Barra rapida: 4 slots de items equipados
- Player.equip(Item) y Player.use(Item)	- Tests de Potion.use(): recuperacion de HP	- Mostrar HP del jugador con barra visual
- Afinidad de armas: bonus por tipo de personaje	- Iniciar GameState: clase contenedora del estado	- Boton "Siguiente fase" para avanzar el turno
ENTREGABLE: El jugador tiene inventario. Los items se pueden equipar. El panel lateral muestra los stats actualizados.		

! Dia reducido: priorizar inventario basico y panel lateral sobre efectos especiales.

DIA 4 — 20 mayo — Martes		Ritmo: COMPLETO
Marino	Pacheco	Castro

- Formula de combate: calcularDanio(atacante, defensor)	- Tests de calcularDanio(): min, medio, max	- Mostrar enemigos en el grid con sprite distinto
- Fase 4 — Ataque del jugador contra enemigo en rango	- Tests de IA Warrior: BFS correcto hacia jugador	- Seleccionar enemigo para atacar (click)
- IA Warrior: BFS hacia jugador, ataca en rango 1	- Tests de linea de vision (Bresenham)	- Mostrar daño infligido con label temporal
- IA Archer: zona de confort, linea de vision (Bresenham)	- Tests de efectos de estado: SLOW reduce movimiento	- Animacion simple: cambio de color al recibir daño
- IA Controlador: aplica efecto SLOW o BLIND	- GameState: serializar sala actual con enemigos	- Log de combate en panel inferior (ultimas 5 acciones)
- executeTurn() para cada tipo de IA		
ENTREGABLE: Combate completo jugable: el jugador ataca, el enemigo responde con IA. El log muestra las acciones.		

DIA 5 — 21 mayo — Miercoles		Ritmo: COMPLETO
Marino	Pacheco	Castro
- LectorGrafoJson.cargarDesdeJson() para el mapa	- Tests de serializacion: guardar y cargar GameState	- Pantalla de inicio: elegir personaje (Kael/Syra/Dorath)
- DungeonGenerator: generar 12 salas con datos fijos	- Tests de DungeonGenerator: 12 salas correctas	- Pantalla de inventario completo con silueta
- Serializar GameState completo a JSON (Gson)	- Tests de excepciones personalizadas	- Boton guardar partida funcional
- Deserializar GameState: reconstruir referencias de objetos	- Verificar integridad de datos tras carga de JSON	- Boton cargar partida funcional
- Manejo de excepciones: InvalidMoveException, etc.	- Tests de TurnManager con enemigos multiples	- Popup al pisar item en suelo
ENTREGABLE: Guardar y cargar partida funciona completamente. La pantalla de inicio permite elegir personaje.		

DIA 6 — 22 mayo — Jueves		Ritmo: COMPLETO
Marino	Pacheco	Castro
- Transicion entre salas: TurnManager.changeRoom()	- Tests de transicion de salas	- Transicion visual entre salas (cambio de sala en UI)
- IA Invocador: invocar Berserkers c/2 turnos	- Tests de drops de enemigos	- Mostrar contador de turnos en pantalla
- Contadores: globalTurnsLeft y roomTurnsLeft	- Tests de contadores de turno	- Pantalla de game over con opcion continuar/nueva partida
- Drops de enemigos al morir: probabilidades	- Integracion completa: juego de punta a punta	- Pantalla de victoria (cuando llega a S5-D)
- Pasillos de transicion: 3x8, 1 item aleatorio	- Tests de regresion: todo lo anterior sigue pasando	- Pulido visual: iconos, colores, fuentes consistentes
ENTREGABLE: Juego completo de punta a punta: inicio, exploracion de 12 salas, combate, guardado, final.		

DIA 7 — 23 mayo — Viernes		Ritmo: COMPLETO
Marino	Pacheco	Castro
- Corregir bugs criticos detectados en prueba completa	- Cobertura de tests >= 70% en clases no visuales	- Corregir bugs de UI detectados en prueba completa

- Optimizar BFS si hay lentitud	- Tests de casos extremos: sala vacia, jugador sin HP	- Probar en resolucion distinta (robustez UI)
- Añadir arbol de decision de IA (para criterio arboles)	- Iniciar borrador de memoria PDF	- Añadir tooltips en items del inventario
- Revisar todas las excepciones personalizadas	- Preparar estructura del diario de uso de IA	- Grabar video de demostracion (borrador)
ENTREGABLE: Version 1.0 jugable y estable. Tests pasando. Borrador de memoria iniciado. Video borrador grabado.		

5.3 Fase B — Entrega final (24-27 mayo)

* El 24 el código está CERRADO. No se añaden funcionalidades nuevas tras esta fecha.

* Los días 25-27 son exclusivamente para memoria, video y pulido de tests.

* El 28 por la mañana es la entrega final.

Día	Fecha	Marino	Pacheco	Castro
8	24 may	Freeze de código. Últimos bugs.	Tests finales. Cobertura $\geq 70\%$.	UI final. Freeze de código.
9	25 may	Sección arquitectura de la memoria.	Sección estructuras de datos de la memoria.	Sección JavaFX + MVC de la memoria.
10	26 may	Sección IA + algoritmos de la memoria.	Sección crítica del proyecto (OBLIGATORIA).	Grabar video final con todos los miembros.
11	27 may	Revisión final de la memoria.	Generar PDF de memoria. Preparar entrega.	Montar video final. Revisión general.
12	28 may	ENTREGA FINAL por la mañana.	ENTREGA FINAL por la mañana.	ENTREGA FINAL por la mañana.

5.4 Recortes pactados para la versión del 24

* Estos recortes NO afectan a la nota. El sistema implementado es el mismo, solo hay menos contenido.

* El enunciado no especifica número de salas ni de armas. Especifica que el sistema funcione.

Elemento	Versión completa (PDF)	Versión a entregar	Impacto en nota
Salas	34 salas	12 salas	Ninguno
Armas	12 armas	6 armas	Ninguno
Subtipos enemigo	8 subtipos	4 subtipos	Ninguno
Efectos de estado	5 efectos	SLOW + BLIND	Mínimo
Mini-bosses	Con atk especial	Enemigo reforzado	Mínimo
Pools aleatorios	Por zona	Ítem fijo por sala	Ninguno
Árbol de decisión IA	Opcional	Implementar día 7	Suma puntos
Dijkstra	Opcional	Implementar si sobra tiempo	Suma puntos

5.5 Metodología de uso de IA — reglas del equipo

* El enunciado evalúa el uso correcto de IA como herramienta. No penaliza usarla, pero sí exige entenderla.

* Regla 1: Nadie sube código al repositorio sin haberlo leído y entendido línea a línea.

* Regla 2: La IA genera borradores. El desarrollador revisa, ajusta y decide.

* Regla 3: Documentar en el diario cada sesión de IA: que se pidió, que salió, que se cambió.

* Regla 4: Los tests los escribe Pacheco manualmente, NO la IA. Así se garantiza que el código se entiende.

* Regla 5: Si la IA genera algo que no se entiende, se descarta y se hace a mano.

Flujo de trabajo recomendado con IA

Pas o	Quien	Que hace	Herramienta
1	Todos	Definir exactamente que clase o funcion se necesita (inputs, outputs, comportamiento)	Guia de diseno v5
2	Marino/Castro	Pedir a la IA que genere el codigo con el spec exacto	Claude / Copilot
3	Marino/Castro	Revisar el codigo generado: logica, estructura, casos extremos	IntelliJ
4	Pacheco	Escribir tests para la clase generada. Si los tests fallan, volver al paso 2	JUnit5
5	Todos	Documentar en el diario: prompt usado, resultado, cambios realizados	Google Docs

5.6 Criterios de exito por dia

Si al final del dia no se cumple el criterio, se escala al dia siguiente y se ajusta el plan.

Dia	Criterio de exito (SI/NO)	Si NO se cumple...
1	La ventana JavaFX muestra un grid. BFS devuelve celdas correctas en test.	Marino prioriza BFS. Castro aplaza panel lateral.
2	El jugador se mueve en pantalla haciendo click.	Castro dedica dia 3 entero a movimiento. Items se aplazan.
3	Los items se pueden equipar y el panel lateral se actualiza.	Simplificar inventario a solo pociones. Armas al dia 4.
4	El jugador puede atacar a un enemigo y el enemigo responde con IA.	Reducir a 1 tipo de IA (Warrior). Archer al dia 5.
5	Guardar y cargar partida funciona sin perder datos.	Aplazar pantalla de inicio. Priorizar JSON sobre UI.
6	El juego es jugable de inicio a fin (aunque sea basico).	Reducir a 6 salas si 12 no estan listas.
7	No hay bugs que impidan jugar. Tests >= 60% cobertura.	Priorizar estabilidad sobre funcionalidades extra.